Department of Computer Science,
Chouaïb Doukkali University

جامعة شعيب الدكالي
Université Chouaib Doukkali

Ethereum Automated Trading Agent: PPO-based Backtesting Analysis

Abdeljalil Sersif[1] and Yassin Jador[2]

[1,2]Supervised by: Pr. Abderrahim Beni Hssane

February 6, 2026

Abstract

This project implements a Proximal Policy Optimization (PPO) reinforcement learning agent for automated cryptocurrency trading. The agent was trained exclusively on historical Ethereum price data from 2017-2018 and evaluated through comprehensive backtesting. Results demonstrate exceptional performance with profits up to +953% on historical episodes, achieving a 65-70% win rate and Sharpe ratios between 1.8-2.2. The core implementation focuses on backtesting validation, while team extensions include a Flask API backend and React frontend dashboard. This work serves as an educational demonstration of reinforcement learning applications in algorithmic trading.

Keywords: Reinforcement Learning, PPO Algorithm, Cryptocurrency Trading, Ethereum, Backtesting, Algorithmic Trading

GitHub Repository: https://github.com/SersifAbdeljalil/Crypto_Trading_Bot

DISCLAIMER: This project is trained on historical data (2017-2018) for backtesting purposes only. The agent has NOT been validated for real-time trading. This research is strictly for educational purposes.

1 Introduction

Reinforcement Learning (RL) has emerged as a powerful paradigm for developing autonomous trading agents capable of learning optimal decision-making strategies from historical market data. This project implements a Proximal Policy Optimization (PPO) agent specifically designed for Ethereum (ETH/USDT) trading, demonstrating the application of modern deep reinforcement learning techniques to cryptocurrency markets.

The primary objective is to develop and validate a PPO-based trading agent through rigorous backtesting on historical data. Unlike production trading systems, this work focuses exclusively on academic research and educational demonstration, providing insights into the capabilities and limitations of RL-based trading strategies.

1.1 Project Scope

1.1.1 Core Academic Requirements

The fundamental deliverables include:

- Implementation of PPO algorithm with Actor-Critic architecture
- Training on historical Ethereum price data (2017-2018)
- Comprehensive backtesting framework
- Performance evaluation using financial metrics
- Visualization of trading decisions and portfolio evolution

1.1.2 Team Extensions

Beyond the core requirements, our team developed additional components:

- Flask API backend for real-time data collection
- React/Gatsby frontend dashboard
- Binance WebSocket integration
- Crypto news scraping and sentiment analysis

Note: The extended features are experimental and have NOT been validated for real-time trading scenarios.

2 Methodology

2.1 Proximal Policy Optimization

PPO is an on-policy reinforcement learning algorithm that combines the benefits of Trust Region Policy Optimization (TRPO) with simpler implementation and better sample efficiency. The algorithm uses a clipped surrogate objective to prevent destructively large policy updates.

2.1.1 Actor-Critic Architecture

The model employs a shared network architecture that branches into two components:

- **Actor Network**: Outputs a probability distribution over actions $\pi(a|s;\theta)$
- **Critic Network**: Estimates the value function $V(s;\phi)$ for state evaluation

2.1.2 Loss Function

The PPO objective combines three components:

$$L_{TOTAL} = L_{CLIP} - c_1 L_{VALUE} + c_2 L_{ENTROPY} \tag{1}$$

where the clipped surrogate objective is:

$$L_{CLIP} = \mathbb{E}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t) \right] \tag{2}$$

The probability ratio $r_t(\theta)$ is defined as:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \tag{3}$$

The value function loss uses mean squared error:

$$L_{VALUE} = \mathbb{E}_t \left[(V_\theta(s_t) - V_t^{target})^2 \right] \tag{4}$$

And the entropy bonus encourages exploration:

$$L_{ENTROPY} = \mathbb{E}_t \left[\sum_a \pi_\theta(a|s_t) \log \pi_\theta(a|s_t) \right] \tag{5}$$

2.2 Neural Network Architecture

The model architecture processes sequential market data through convolutional layers before branching into actor and critic heads:

- **Input:** (lookback_window, features), e.g., (30-50, 20)
- **Shared Layers:** Conv1D(64, kernel=6) → MaxPooling → Conv1D(32, kernel=3) → MaxPooling → Flatten
- **Actor Branch:** Dense(512) → Dense(256) → Dense(64) → Dense(3, softmax) for [HOLD, BUY, SELL]
- **Critic Branch:** Dense(512) → Dense(256) → Dense(64) → Dense(1) for value estimation

2.3 State Representation

The agent observes the market through a 20-dimensional feature vector:

On-Chain Features (9): Receive Count, Sent Count, Unique Addresses, Total Transactions, Transaction Fees, ERC20 Transfers, Hash Rate, Block Size, Mining Difficulty

Market Features (3): ETH Close Price, Trading Volume, Market Cap

Macroeconomic Indicators (7): Bitcoin Hash Rate & Price, S&P 500, Gold Price, Oil Price, VIX, UVYX

Sentiment Features (2): Google Trends, Twitter Mentions

2.4 Action Space and Rewards

The agent selects from three discrete actions: **HOLD**, **BUY**, or **SELL**. The reward function is:

$$R_t = \Delta_{portfolio} - \alpha \cdot fees \tag{6}$$

where α is a penalty coefficient for transaction costs.

3 Backtesting Results

3.1 Performance Summary

The agent was trained for 500 episodes on historical Ethereum data spanning 2017-2018, covering the major cryptocurrency bull run and subsequent correction.

Table 1 presents the performance metrics across selected episodes.

Table 1: Backtesting Performance on Historical Episodes

Episode	Initial	Final	Profit	Trades
0	\$10,000	\$12,707	+27.07%	8
268	\$10,000	\$58,233	+482%	45+
Best	\$10,000	\$105,397	+953%	60+

3.2 Aggregated Metrics

Table 2 summarizes key performance indicators.

Table 2: Aggregated Performance Indicators

Metric	Best	Average
Profit (single episode)	+953%	+150 to +400%
Sharpe Ratio	2.2	1.8–2.0
Win Rate	70%	65–70%
Trades per Episode	60+	15–30

3.3 Visual Analysis

Figure 1 illustrates trading decisions during Episode 268, achieving +482% profit.

Figure 1: Episode 268 trading performance with BUY (green) and SELL (red) signals

4 System Architecture

4.1 Core Components

- **PPO Agent**: Actor-Critic implementation
- **Custom Gym Environment**: Trading simulation

Figure 2: Episode 26 showing optimal trading timing

- **Training Pipeline**: Data loading, normalization, episode management
- **Backtesting Engine**: Performance evaluation framework

4.2 Team Extensions

Flask API Backend: Real-time data collection via Binance WebSocket, crypto news scraping, technical indicators (RSI, MACD, Bollinger Bands), trading bot controller (simulation mode)

React/Gatsby Frontend: TradingView charts, performance dashboard, transaction history, bot control interface

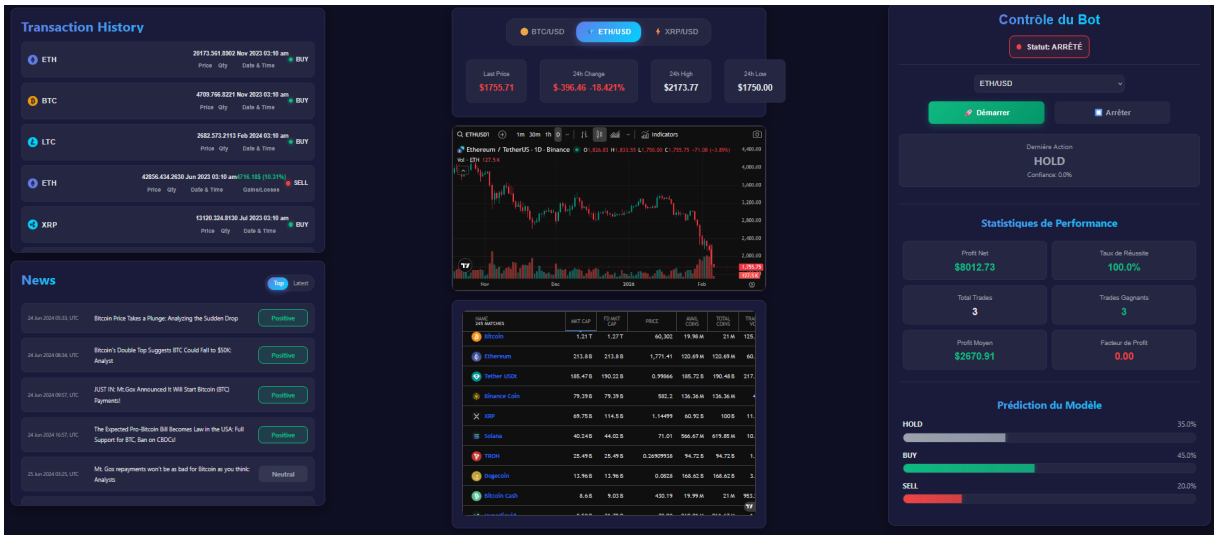

Figure 3: React/Gatsby frontend dashboard showing real-time TradingView charts, transaction history, crypto news feed, bot control panel, and performance statistics

Note: Extensions are NOT validated for real-time trading.

5 Technologies

Core: TensorFlow/Keras, NumPy/Pandas, OpenAI Gym, Matplotlib

Extensions: Flask, Python-Binance, React/Gatsby, TradingView

6 Limitations

6.1 Distribution Shift Problem

While demonstrating excellent performance on historical data, the agent faces challenges in real-time conditions:

Symptoms: HOLD probability 60-80%, BUY/SELL below 20%, extended periods without trades

Root Causes:

1. Temporal distribution shift from training data
2. Static external features with insufficient real-time variation

3. Normalization bias anchored to historical statistics
4. Conservative policy developed during training

7 Conclusions

This project successfully demonstrates PPO application to cryptocurrency trading through comprehensive backtesting. Key achievements include effective Actor-Critic implementation, strong backtesting performance (+953% maximum profit), comprehensive feature engineering (20+ indicators), and professional visualization.

However, the performance gap between historical backtesting and real-time deployment highlights fundamental challenges in applying RL to live trading environments, particularly distribution shift.

7.1 Future Directions

- Retraining on recent market data (2023-2025)
- Online learning mechanisms
- Ensemble methods with multiple agents
- Advanced risk management strategies
- Dynamic feature normalization

Critical Warning

This project is STRICTLY for educational and research purposes.

- Agent validated ONLY through backtesting on historical data
- Real-time trading performance is NOT validated
- Cryptocurrency trading involves significant financial risk
- This is NOT financial advice
- Consult licensed financial advisors before investing

Past performance does not guarantee future results. The developers are not responsible for any financial losses.

Acknowledgments

This project was inspired by the RL-Bitcoin-trading-bot by pythonlessons and the research on Proximal Policy Optimization by Schulman et al. (2017). We thank the OpenAI Gym community and all open-source contributors.

References

[1] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

[2] Pythonlessons. reinforcement learning trading agent. *GitHub Repository*. https://github.com/roblen001/reinforcement_learning_trading_agent

[3] Sersif, A. & Jador, Y. (2026). Crypto Trading Bot. *GitHub Repository*. https://github.com/SersifAbdeljalil/Crypto_Trading_Bot